Deploying Hello World App from Local Docker to AWS EKS

1. Introduction

In this guide, we will build a simple Hello World application in Python, containerize it with Docker, push it to AWS ECR (Elastic Container Registry), and deploy it on AWS EKS (Elastic Kubernetes Service).

2. Create Hello World App Locally

Step 1: Create the Python Code

```
# hello.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return "Hello, World from EKS!"

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=5000)
```

Step 2: Create the Dockerfile

```
# Dockerfile
FROM python:3.9-slim

WORKDIR /app
COPY hello.py .
```

```
RUN pip install Flask

EXPOSE 5000

CMD ["python", "hello.py"]
```

Step 3: Build and Run the Docker Image

```
# Build the Docker Image
docker build -t hello-world-container .

# Test Locally
docker run -p 5000:5000 hello-world-container

# Access via Browser
http://localhost:5000
```

3. Push Docker Image to AWS ECR

```
# Create ECR Repository
aws ecr create-repository --repository-name hello-world-container --region us-east-1

# Login to ECR
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <aws_account_id>.dkr.ecr.us-east-1.amazonaws.com

# Tag the Image
docker tag hello-world-container:latest
```

<aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/hello-world-container:latest

Push the Image to ECR

docker push <aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/hello-world-container:latest

4. Deploy the App on AWS EKS

Step 1: Update Kubernetes Deployment

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hello-world
        image: <aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/hello-world-container:latest
        ports:
```

```
        - containerPort: 5000
```

Step 2: Create Kubernetes Service

```
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: hello-world-service
spec:
  selector:
    app: hello-world
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
  type: LoadBalancer
```

Step 3: Apply Kubernetes Manifests

```
# Apply Deployment and Service
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

# Verify Deployment
kubectl get pods
kubectl get svc hello-world-service

# Access the App
```

http://<external-ip>

5. Conclusion

You have successfully built a Python-based Hello World application, containerized it with Docker, pushed the image to AWS ECR, and deployed it on AWS EKS. This process showcases the complete workflow of modern cloud-native deployments.